SRED: Stabilized RED

ns-3 Implementation Project Proposal

ID: 2105020

This project is based on the SRED paper by Ott, Lakshman, and Wong, which improves RED without maintaining per-flow state.

Paper link :

https://www.teunisott.com/Papers/Lakshman/Infocom1999.pdf

The Congestion Control Challenge

Standard queue management mechanisms struggle to maintain stability when the number of active flows varies significantly.

Background: RED vs. SRED

Random Early Detection (RED)

RED drops packets preemptively based on average queue length. However, its effectiveness degrades when the number of active flows (N) is high, leading to high packet loss or full buffers. RED is highly sensitive to threshold and probability parameters, which are difficult to tune under dynamic traffic loads.

Stabilized RED (SRED)

Stabilized RED (SRED) introduces a statistical method to estimate the number of active flows (N) without maintaining per-flow state. This estimate is then used to adapt the packet drop probability, allowing the queue to remain stable under varying traffic loads.

The Zombie List

Stateless Flow Tracking

SRED relies on a "Zombie List," a short-term cache (Size M) of recently seen flows.

Instead of maintaining state for every flow, it stores minimal data: Flow ID, Count, and Timestamp.

This allows SRED to scale efficiently without the memory overhead of per-flow queuing.

Hit Detection & Estimation

- **Hit Detection:** Every arriving packet is compared to a randomly chosen "Zombie" from the list.
- **Hit:** If Flow IDs match, it's a "Hit".
- **Miss:** If they differ, the Zombie is overwritten with probability *p*.

Hit Frequency P(t)

$$P(t) = (1 - \alpha)P(t - 1) + \alpha Hit(t)$$

Where α is a weight factor (e.g., 1/M).

The Mathematical Link

$$\frac{1}{P(t)}$$

Approximated Active Flows (N)

Why this matters

The inverse of the hit probability provides a surprisingly accurate estimate of the effective number of active connections (N) without complex state tracking.

Stabilizing the Drop Probability

The Mechanism

Standard RED drops based only on Queue Length (q).

SRED scales the drop probability based on the estimated number of flows.

This ensures that as N increases, the drop probability rises aggressively enough to keep the buffer from overflowing.

Zap Probability

$$p_{zap} = p_{sred}(q) \times min(1, \frac{1}{(256 \times P(t))^2}).$$

Where Psred is a function on drop probability

Identifying Misbehaving Flows

Identify

Flows that generate a disproportionate number of "Hits" in the Zombie list are likely taking more than their fair share of bandwidth.

Verify

We calculate "Total Occurrence" (Count + 1) for suspected flows to confirm if they are statistical outliers.

Penalize

Once identified, SRED can apply aggressive drop probabilities specifically to these "bandwidth hogs" to protect fair flows.

Proposed ns-3 Implementation

- **New Class:** Create SredQueueDisc derived from the standard QueueDisc class in ns-3.
- **Data Structure:** Implement a lightweight ZombieList class to manage flow IDs and timestamps.
- **Logic Override:** Override the DoEnqueue method to perform the Hit/Miss comparison logic before accepting packets.
- **Moving Average:** Implement the P(t) calculation updated on every packet arrival.
- **Attributes:** Expose tunable parameters: List size (M), Weight (α), and Max Drop Prob (p_max).

Modification : Delay-Aware Drop Probability

Limitation

Original SRED calculates drop probability based solely on **queue length.**This ignores the **latency** experienced by packets, which is critical for real-time applications.

Proposed Solution

Integrate Average Queue Delay into the decision logic.The effectiveness will be evaluated using average queue delay, packet drop rate, and throughput in ns-3 simulations.

Delay Adjustment

$$D_n(t) = \min\left(1, \frac{D(t)}{D_{ref}}\right)$$

Where D(t) is EMWA of delay

Delay Aware Zap Probability

$$p_{DA}(t) = p_{sred}(q) \cdot \min\left(1, \frac{1}{(256\,P(t))^2}\right) \cdot \left(1 + \gamma\,[D_n(t)]^k\right)$$

Where gamma is a weight factor and

some k>0 to penalize large delays more aggressively